

* Bags = duplicates

- Projection (π)** = pick cols
 - SELECT id, citation FROM stops
 - Ex: $\pi_{id, citation}(stops)$
- Selection (σ)** = filter rows by cond.
 - SELECT id, citation FROM stops WHERE age < 50
 - Ex: $\sigma_{age < 50}(stops)$
- Renaming (ρ)**
 - SELECT id AS person_id FROM stops AS stops_data
 - Ex: $\rho_{stops_data(id \rightarrow person_id)}(stops)$
- Cartesian product ($\times$)**
 - merge col. from 2 relations + pairwise combo.
 - SELECT * FROM stops, zips
 - Ex: $\sigma_{stops.location = zips.location}(stops \times zips)$
- Theta Join ($\bowtie$)** $\leftarrow$ doesn't drop rows
 - Cartesian product + selection condition
 - SELECT * FROM a INNER JOIN b ON a.col1 > b.col2
 - Ex: $a \bowtie_{a.col1 > b.col2} b \quad R_1 \bowtie_{R_1.R_2 = R_2.R_1} R_2$
- Equi Join**
 - theta join w/ equality condition
 - SELECT * FROM stops INNER JOIN zips ON stops.location = zips.location
 - Ex: $stops \bowtie_{stops.location = zips.location} zips$
- Natural Join ($\ltimes$)** $\leftarrow$ filter rows
 - 2 relations share attribute name + type
 - SELECT * FROM stops NATURAL JOIN zips
 - Ex: $stops \ltimes zips$

- Other Cases**
 - R WS = RNS w/ same schema
 - R MT = R x T w/ no same attributes
 - Union, $\cap$ intersection, - difference
 - Ex: $\pi_{name}(students) \cap \pi_{name}(professor)$
- Ex: SELECT a.name, b.instructs FROM student a, teachers b WHERE a.class = b.instructs AND b.area = "ds"
- $\pi_{name, instructs}(\sigma_{class = instructs}(student \times \sigma_{b.area = "ds"}(teachers)))$

- SQL**
 - pre-aggr, post-aggr
- Execution:** FROM $\rightarrow$ WHERE $\rightarrow$ GRP BY $\rightarrow$ HAVING $\rightarrow$ SELECT $\rightarrow$ ORDER BY $\rightarrow$ LIMIT
- NULL** is filtered out by WHERE unless you use IS NULL or IS NOT NULL
- GROUP BY:** every col. in SELECT is agg or in GRP list
- CASE WHEN** = inside SELECT or agg
- Ex: SELECT AVG(CASE WHEN loc = 'downtown' THEN age END) AS avg_dt_age

- Joins**
 - left = all left rows, nulls on right (vice versa)
 - outer = all rows both, nulls on both
 - inner = rows that match cols.
 - cross = cartesian prod., all row combos
 - natural = join on common cols (inner join default)
- Agg**
 - collapse grouped rows into single row
 - COUNT(*) includes NULL
 - COUNT(col-name) excludes NULL
 - SUM, AVG, MIN/MAX ignore NULL

Union All
Union = no dupl. rows

3 CTE, VIEWS, QUERIES

	Table	CTE	(Virtual) VIEW	Materialized view
Storage	disk	memory	memory	disk
revised?	Y	N	Y	Y
when does query get computed?	N/A	once defined, used + discarded	when view is referenced	when created (maybe auto refreshed)
use case	store data, long-term	readable complex query, 1 time use	save query, table update freq.	table not update, little recompute, save result

- CREATE VIEW citation_stops AS (SELECT gender, citation FROM stops WHERE citation = True);
- SELECT gender FROM citation_stops;
- CREATE MATERIALIZED VIEW citation_stops AS (...);
- CTE: WITH citation_stops AS (...)
- SELECT * FROM citation_stops;
- Scalar subqueries
 - Subquery return single tuple w/ single attribute as scalar value
 - SELECT name FROM employees WHERE salary > (SELECT AVG(salary) FROM employees);
- EXISTS + NOT EXISTS in WHERE to filter row existence
- SELECT DISTINCT stops.location FROM stops WHERE NOT EXISTS (loc in stops who corresp. location in zips)
- (SELECT * FROM zips WHERE zips.loc = stops.loc)

4 WINDOW FUNCS

- use value from many rows $\rightarrow$ value per row
- agg $\rightarrow$ a value per grp/partition
- cumulative sum, ranks, etc. over a set of rows
- (1) Grp rows w/ PARTITION BY so func calculates w/in grps
- (2) sort rows w/in frame w/ ORDER BY (for i.e. ranks, running sum, avg)
- (3) specify RANGE/frame's relation to current row $\rightarrow$ req ORDER
- Compute function $\rightarrow$ over a window $\rightarrow$ where tuples are ordered like so
- $\rightarrow$ take this frame of interest in window
- Ex: Find cumulative total spending up to/including that order
- SELECT student_name, order_id, SUM(price) OVER (PARTITION BY student_id ORDER BY order_id) AS cum-total-spend

5 Strings

- REGEXP_REPLACE(source, pattern, replacement, flags)
- REGEXP_REPLACE('Hannah Montana', '(.)', '\1 \1') $\rightarrow$ LN FN
- SUBSTRING (string, start, length) $\rightarrow$ one-index
- SUBSTRING (string, pattern) $\rightarrow$ corres. to pattern
- STRPOS (string, substring) $\rightarrow$ one-index
- || concatenates

6 Table Sample/Sampling Rows

- ORDER BY RANDOM() $\rightarrow$ full shuffle, slow
- TABLESAMPLE BERNOULLI(p) $\rightarrow$ fast, flip coin per row
- TABLESAMPLE SYSTEM(p) $\rightarrow$ fast, test, less random, pg-lvl sampling

7 DML + DDL

- INSERT INTO table VALUES (<values that match schema>)
- Ex: stops (id, race, age, location)
- $\rightarrow$ INSERT INTO stops VALUES (5000, 'Asian', 24, 'West Oakland');
- $\rightarrow$ "stops (id, age, location)"
- VALUES (5000, 24, 'West Oakland') $\leftarrow$ race = null
- $\rightarrow$ no values specified $\rightarrow$ location NULL
- $\rightarrow$ INSERT INTO table (<subquery>)
- UPDATE Relation SET <col assignments> WHERE <condition>
- Ex: UPDATE stops SET age = 18, location = 'Rockridge' WHERE age = ...
- DELETE FROM <table-name> $\leftarrow$ all relations
- DELETE FROM <table-name> WHERE <condition>
- Ex: DELETE FROM stops WHERE age < 10
- CREATE TABLE <table-name> (<column name> <column type>, ...)
- DROP TABLE (IF EXISTS) <table>
- ALTER TABLE <table> ADD <col1> <col1 type> DROP <col1>

8 Referential Integrity

- key = no tuples can have same values
- PK vs unique
 - 1 PK only = unique identifier, can be multi cols
 - can be many UNIQUE constraints
 - no attribute of PK can be NULL
 - UNIQUE attributes can be NULL
 - several tuples w/ NULL values for UNIQUE attributes is ok
- Ex: CREATE TABLE stops (...)
- location VARCHAR(20) NOT NULL
- arrest BOOLEAN DEFAULT FALSE
- PRIMARY KEY (stop_id)
- UNIQUE (person_id, stop_time)

- Foreign key (FK)
 - field in one table refers to PK in another
 - Ex: CREATE TABLE Actor (name TEXT actor_id INT PRIMARY KEY)

- CREATE TABLE cast_info (movie_id INT FOREIGN KEY REFERENCES Actor (actor_id))
- FK constraints prevent insertions/deletions that would destroy their links
- $\rightarrow$ prevent insert/update to cast_info if not in actor
- $\rightarrow$ deletion/update to actor $\rightarrow$ cast_info tuple change

9 Cascade

- make the same changes to orders as in students
- ALTER TABLE orders DROP FOREIGN KEY (student_id)
- ALTER TABLE orders ADD FOREIGN KEY (student_id) REFERENCES student (student_id)
- ON DELETE CASCADE: delete order tuples that refer to student deleted in students
- ON UPDATE CASCADE: update to students' student_id col $\rightarrow$ violation as orders' student_id not updated

10 Bits/Memory Model

- CPU runs programs & data stored in...
 - main memory: close to CPU, short-term/volatile, small buffer storage, expensive, fast, RAM
 - Disk: peripheral device, long-term, stable, large, slow, SSD, HDD
- Scanning cost = Disk I/O
- Scan = DS gets each pg from disk, one at a time, & load into main memory for reading/writing
- pages stored in disk $\rightarrow$ load into main memory w/in main memory buffer
- Cost in processing queries = cost of disk I/O = # pg loads
- Read/write = pg load from disk to memory (write back)
- Storage
 - $K \rightarrow M \rightarrow G \rightarrow T \rightarrow P \rightarrow E \rightarrow Y$
 - $10^3 \rightarrow 10^6 \rightarrow 10^9 \rightarrow \dots$
 - data laid out on pages on disk storage
 - page size = 8kb

11 Performance Analysis

- Sequential scan
 - Ex: SELECT * FROM titles WHERE title_id = "112345"
 - w/ 10,000,000 records stored across 500,000 pages
 - load each pg into memory & check if ea. record matches
 - unordered $\rightarrow$ 500,000 pg loads worst-case $\rightarrow$ vce index
- Scan nodes + wmy
- Index (only) scan
 - small amt pgs match
 - index-only: index had all cols
- Seq. scan
 - predicate touch too many pgs + index lookup not worth
 - range predicate $\rightarrow$ seq scan
- Bitmap heap scan
 - which heap pg to visit using index
 - read pgs in batches
 - results land on many of same pgs

VACUUM ANALYZE = all tables
VACUUM (FULL, ANALYZE) tablename = reorg, mv data around

12 Index Selection

- declarative = tell DBMS what rewrites/
- optimize w/ query optimizer, user redesigns/enabling strats/index
- Indexes
 - speed reads on key(s) w/ less scanning
 - automatic for PK & UNIQUE
 - w/ B+ trees = bot nodes - scan nodes, high nodes - join/agg/sort & hash tables
 - CREATE INDEX <idx-name> ON <table> (<cols>)
 - DROP INDEX <idx-name>
- cardinality = # of distinct values in attribute
- cluster data based on index $\rightarrow$ sort + store that order on disk
- CLUSTER <table> ON <idx-name>
- Index
 - Yes: large relation, time-consuming seq-scan, join present, freq. query w/ that attribute, high cardinality, WHERE, key attributes, clustered attr., inner relation of nested + sort
 - No: low cardinality attr. (e.g. T/F), many tuples per value of attr., table modified more than queried
- Ex: Query Optimization
- CREATE TABLE students: 1,000 pgs, 1 mil tuples, 90% > 2.0 GPA, 50% > 3.0 GPA
- SELECT * FROM students WHERE GPA > 3.0 OR name = 'John' } read 1,000 pgs b/c dk name = 'John' where GPA > 3.0 is
- If we index on GPA: 1 GPA < 2.0 - 1 mil $\times$ (1-0.9) = 0.1 mil
- 2 GPA < 3.0 - 1 mil $\times$ (1-0.5) = 0.5 mil

13 Optimization

- predicate pushdown = preselect rows before join
- projection pushdown = preselect cols before join
- Query Execution Plan
- SQL logic to extended RA $\rightarrow$ rewriting $\rightarrow$ cost estimate $\rightarrow$ feed plan
- Ex: SELECT * FROM ... INNER JOIN ... ON ... WHERE ... AND ...
- if this was 0, we have GRP BY/Agg in SELECT
- most optimal b/c all pushdowned
- Diagram: Ma, Scan, Join, Std, Std Sport
- Ex: student_athletes table w/ 100 pgs, 100,000 tuples & students table
- student_name = index; std_ath_name, sport = 2 indexes
- SELECT std_ath.sport, AVG(GPA) FROM std_ath INNER JOIN std_students ON std_id = std_ath_id WHERE std_ath.sport = 'soccer' GROUP BY std_ath.sport
- 1 Filter sport = 'soccer' & index scan std_ath.sport
- 2 Join std_ath & students
- 3 group by std_ath.sport

14 Join Types

- Nested loop join: every tuple R x S, check if it matches
- Sort-merge: 1 sort R on join attr. & for s 2 run down + merge matching
- Hash join: 1 R, S hash into buckets by attr. 2 read both + join
- nest-loop + hash: join predicate is not equi-join (=) & small table
- Sort-merge + hash: output on sorted cols
- Hash + nested loop: large table & nested too long

15 Models

- matrix/tensor rank 2: type M x N
- array of vals sparse
- symmetric
- transpose w/o info loss N x M
- tensor rank 3: type M x N x O
- speedy coord of array
- 0, scalar; 1 vector
- relation/table: name | name type | type
- schema: name $\leftrightarrow$ data type
- sets of tuples
- relation $\rightarrow$ matrix + lose info (rows)
- matrix $\rightarrow$ relation: keep info
- PF: name | name type | name type
- Arrn array of vals
- relation + matrix
- index ordering + named vals/cols

16 Data Prep

- granularity = lvl detail in data
- roll-up: $\rightarrow$ coarser (1) $\rightarrow$ remove attr. in GRP-BY
- drill-down: $\rightarrow$ finer $\rightarrow$ GRP BY more or sample
- hierarchy dictates transformation
- explicit, time, space
- is-a or predicate/relation

- Outliers
 - removal: needs center (avg, med) + dispersion (std, var, iqr)
 - handling: outlier influences dispersion + outlier detection
- Gaussian
 - mean, stddev, z-score
 - remove $(z\text{-score})^2 \geq 2$ = keep 2.5% - 97.5% percentile
- Trimming
 - median = more robust to outliers
 - remove fixed percentage
- Winsorizing $\leftarrow 1\% = 1^{st} - 99^{th}$
 - replace outlier value w/ value just before removed
 - tail of repeated values
- How robust outlier removal is to intro. of corrupt values
 - Breakdown point = smallest fraction of corr. values before incorrect result
 - BPP of mean = 0
 - 1% trimmed mean = 1% median = 50%
- MAD(x) = median(|x_i - $\bar{x}$ |)
 - 1SD = 1.4826 MADs $\rightarrow$ Hampel x84: filter 2x1.4826 MADs

Imputation (NULL)

- default col. val. w/ col. agg
 - corr. across col = cond. distribution given other col data
 - general model-based interpolation
 - interpolation across ordered rows
- Ex: SELECT att_id, CASE WHEN att.num_std IS NOT NULL THEN att.num_std ELSE (a.agg) END
- FROM disc_metrics
- 5x: (linear regression: (a slope * a date + a intcp)::int WITH (avg) AT SELECT id, regr_slope(num_std, date) AS slope regr_intcp(num_std, date) AS intcp FROM disc_metrics GROUP BY id)
- 1 index

Linear interpolation

- consecutive NULL = interpolate w/ scalar math
- | X | Y | run | run_start | next_val | run_end | run_size | run_rank |
|---|---|-----|-----------|----------|---------|----------|----------|
| 1 | 1 | 1 | 1 | 4 | 5 | 1 | 1 |
| 2 | 1 | 1 | 1 | 4 | 5 | 1 | 2 |
| 3 | 1 | 1 | 1 | 4 | 5 | 1 | 3 |
| 4 | 4 | 2 | 4 | 5 | 5 | 1 | 1 |
| 5 | 5 | 2 | 4 | 5 | 5 | 1 | 2 |
| 6 | 5 | 2 | 4 | 5 | 5 | 1 | 3 |
| 7 | 7 | 3 | 7 | 7 | 7 | 1 | 1 |
| 8 | 7 | 3 | 7 | 7 | 7 | 1 | 2 |
| 9 | 7 | 3 | 7 | 7 | 7 | 1 | 3 |
- 3 pass
 - Forward: run, run_start, get next_val into last row of each run
 - Backward: partition by run for run_size/rank, run_end from next_val
 - Scalars to interpolate unordered

Keys & Updates

- PK = UNIQUE, NOT NULL, identify rows, parent
- FK = match a PK or be NULL, duplicates OK
- UNIQUE (date, time) $\rightarrow$ no 2 rows w/ same combo
- NOT NULL $\rightarrow$ can't set to NULL
- Delete from child table
- Update non-key fields
- Insert duplicate PK or invalid FK
- Delete/update parent PK if referenced X

Q3: add column (simpler, expensive?)

ALTER TABLE ideas ADD COLUMN votes INT DEFAULT 0

- 4.4 • 5000 tuples in users, 4-10 members in users
- 500 tuples in teams $\rightarrow$ 10% are invalid users
- 1000 tuples in matches

- Right Join users onto teams: $5000 + 500 - 0.1 = 5050$ rows
- Inner Join users w/ itself: $500 \times 10 - 10 = 5000$ rows on team_id, 1500 teams, 10x10 members

4.5

- mutual index for AND
- separate index for OR sides

SP 25 MT

1.1 Impose constraint

ALTER TABLE players ADD CONSTRAINT new_constraint

UNIQUE (game_id, player_name)

1.2 CREATE VIEW positive_msg AS

SELECT * FROM messages
WHERE (message LIKE '%good game%' OR message LIKE '%thank you%')
AND created_at BETWEEN '2024-01-01' AND '2024-12-31'

Calculate positive percentage:

SELECT ..., FLOAT(COUNT(p.id)/(SELECT COUNT(*) FROM... WHERE...)) * 100 AS...
FROM games AS g LEFT JOIN...

game honor list to highest reputation score:

SELECT p.p_id, p.p_name, p.rep_score,
RANK() OVER (ORDER BY p.rep_score DESC) AS rank
FROM players AS p INNER JOIN games AS g ON...
WHERE... ORDER BY rank

banned but rep_score > 50

SELECT id, name FROM games AS g
WHERE EXISTS (SELECT * FROM players AS p
WHERE p.g_id = g.id AND status = 'banned'
AND p.rep_score > 50)

2.1 SELECT SUM() FROM
WHERE ... AND ... AND ...
GROUP BY... HAVING...

2.2 SELECT o.o_id, r.name, o.amt
FROM order o, retri r
WHERE r.r_id = o.o_id AND o.amt > r.min_amt
AND o.amt = 50

Tip: SELECT a.name
FROM books b JOIN authors a ON b.a_id = a.a_id
WHERE b.year = 1974

name (year=1974 (books)) b.a_id = a.a_id (authors)

- Join w/ filter: condition (ROWS)
- Filter by Join: condition (ROWS)
- agg COUNT: $\pi_{key}(COUNT > k)$ ($\sigma_{key, COUNT(c)}$ (R))
- Either...or: use v in σ
- BOTH...AND: $\wedge$ in σ
- exactly one/none: -
- union of results: $\cup$
- interdiction: $\cap, R - (R - S)$
- $\pi \rightarrow \sigma \rightarrow \Join \rightarrow \Join \rightarrow \Join$ (scan)

5.1 Staff id (4 bytes per row) = $1000 \times 4 = 4000$ bytes

5.2 $\log_2(1000)$ -bit binary code $\rightarrow \log_2(1000) \times 8 = 67.7 \approx 68$ bits

5.4 unique staff member now 500 $\rightarrow \log_2(500) \times 8 \rightarrow$ dedupe effective

5.5 calculate each staff member's ranking per game by points

SELECT game_id, points,
RANK() OVER (PARTITION BY game_id ORDER BY points DESC) AS...
FROM... ORDER BY...

5.6 Rank performance $\rightarrow$ diff in rank

WITH ranked_performance AS (...)
SELECT r.game_id, r.diff_id, ...,
LAG(r.rank_in_game) OVER (PARTITION BY r.staff_id
ORDER BY r.game_date) AS prev_rank,
r.rank_in_game,
r.rank_in_game - LAG(r.rank_in_game) OVER " "
FROM ranked_performance AS rp...

- ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
- RANGE BETWEEN CURRENT ROW AND UNBOUNDED PRECEDING

MT FA24

- CHAR(n) = n bytes - timestamp = 8
- BOOL = 1 byte - epoch time = 4
- INTS = 2, 4, 8
- FLOAT = 4, 8

779-6-543-21012-3
 $\rightarrow$ REPLACE (11bn, '-', '')::INTEGER replace
 $\rightarrow$ REGEXP_REPLACE (11bn, '-', 'g')

table - σ - π - $\Join$

WHERE return_date IS NULL $\leftarrow$ not returned = checked out

user_name, threadsPosted
display_name, COUNT(*)
user_id = threads.user_id

rows in result sorted if...

index scan, index only scan

sort
sort key: created_at

thread_type can be 3 things
 $\Rightarrow$ need 2 bits: 4 patterns to represent

for n categories, you need $\log_2(n)$ bits

Predicate pushdown:

SELECT * FROM ord o JOIN
SELECT * FROM cust WHERE country = 'US' AS c
ON o.c_id = c.c_id

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

instead of SFW

$$\sqrt{\frac{\sum (x_i - \bar{x})^2}{n}}$$

n-1 sample
n population

MAD = median (|x_i - median(x)|)

2 5 4 3 2 1

multiply X integer by 8

X = X << 3

left shift by 3 multiplies X by 8 b/c $2^3 = 8$

X = (X >> 3) << 3

right, left = round down

X = ((X - 1) >> 3) << 3 + 8

round up

- Entity set $\square$
- attribute $\bigcirc$ unique
- relationships $\diamond$
 - at most one $\rightarrow$
 - at least one $\leftarrow$
 - exactly one $\rightarrow$
 - zero or more $\dashrightarrow$

- Normalizatiion
 - split + decompose relations into multiple
 - less redundancy + update/delete anomalies
 - Functional Dependency
 - constraint btwn 2 sets of attributes
- ID | Name | Title | Hours per week
 $\rightarrow$ Title | Hrs per week & ID | Name | Title

- Semistructured Data
 - structured = matrices/relations/dfs
 - fixed # attr. per row
 - atomic = value of same col. type
 - flat = 1 value per cell
 - semistructured
 - flexible, multivalued, nested
 - NoSQL
 - large service, non-relational, horizontal scalability, low consistency needs
 - financial + data consistency, load+balance+ transaction history, relationships $\rightarrow$ SQL db
 - JSON
 - human readable, tree structure
 - primitive, objects (K,V), or arrays
 - no joins support
 - Nesting $\rightarrow$ redundancy + denormalization
 - flexible attributes, denormalize/avoid join, self-describing
 - update/delete anomalies, redundancy, hard to find inner-nested info, less compact
 - XML
 - Xpath query language + Xquery
 - tree-structured ... $\langle / \rangle$

- Parallel/Distributed Computing
 - task divided into subtasks executed concurrently in single (parallel) or multi (distributed) machines
 - skew = uneven data distribution
 - Horizontal Partitioning
 - Range-Based: label bucket range, skew likely
 - Hash-Based: define hash function, skew depend
 - Round-robin: hash % n, no skew but more work
- CAP Theorem
 - data store delivers at most 2
 - Consistency: only show correct data, same data
 - Availability: will give data but may be old node
 - Partition-tolerance: cluster continues to work despite communications breakdown
 - CP $\rightarrow$ shut non-consistent node until partition resolve
 - AP $\rightarrow$ old nodes/stale reads during partition
 - CA x

- Conflicting actions IF
 - 2 diff. concurrent transactions
 - reference same object
 - at least one is a write
- $\rightarrow$ T1, T2 Conflicts, then every cf. serial schedule have spec order (serializable)

- Map-Reduce
 - data $\rightarrow$ chunks, across machines (nodes) for parallel process + combine
 - framework / programming model
- MAP (K,V) $\rightarrow$ $\langle K', V' \rangle$
 - one map func call per (K,V) pair
 - Item-by-item, read a lot
 - shuffle
 - to group by keys for reduce
- Reduce (K', $\langle V' \rangle$) $\rightarrow$ $\langle K', v'' \rangle$
 - values w/ same key reduced/processed together
 - one func call per unique key K'

- DBT: Data Build Tool
 - declarative specification framework for pipelines of ETL transform
 - data quality tests
 - set derived views base on base relation
 - define unit tests
 - reference using FROM
 - dbt init: initialize proj.
 - dbt run: run models, transform/create views/tables
 - dbt test: check schema/data integrity

- Sampling
 - TABLESAMPLE BERNOULLI: row-level
 - TABLESAMPLE SYSTEM: page-level
 - Goal: K-sized SRS where every record has 1/n probability in sample
 - sample $\rightarrow$ smaller data for pipeline / less compute
 - DON'T JOIN 2 sample tables
 - sample $\rightarrow$ faster query / approximation
 - inaccurate, bias, uncertainty x

- Reservoir Sampling
 - K-sized sample from population of n elem $\leftarrow$ K chance in reservoir
 - SRS iid w/o replacement
 - linear runtime, single pass
- Fill reservoir array w/ first K elem
- Scan table. For ith elem,
 - roll random # j from 0 to i
 - j < K: replace jth pos. elem
 - j > K: ignore

- Stratified Sampling
 - split pop into groups $\rightarrow$ sample $\rightarrow$ recombine samples
 - proportionally or w/ weights

- Transactions
 - DBS updates want concurrency control + recovery
 - Transaction: unit of work, should happen together
 - BEGIN UPDATE ... COMMIT as TCL commands

- ACID
 - Atomicity: 'all or nothing' operations of transaction execution
 - Consistency: if COMMIT succeeds, db integrity checks hold
 - Isolation: concurrent transactions externally appear to run sequentially & not see ea. other result
 - Durability: committed transaction $\rightarrow$ permanent in db

- Serializable schedules
 - Serial schedule = actions from diff transactions interleaved
 - no concurrency b/c actions executed together
 - unserializable: no serial equivalent, transactions not isolated

- Serializable = no cycles
- * RR ignore * RW, WW, WR check

- Business Intelligence
 - OLAP (Online analytical processing)
 - large volumes of data
 - read/submarize lots of multidim data
 - read only, many sources
 - long response times
 - large storage req.
 - star/snowflake schema
 - Ex predict consumer behavior - total sales of ea. Honda car over time per city
 - OLTP (Online transactional processing)
 - manage/process real time transaction
 - read, insert, update, delete data in RDB
 - read-write queries
 - shorter response
 - smaller storage
 - normalized/denormalized
 - Ex processing payments/customer data

- Data cube/OLAP cube
 - range partition
 - Cross-tab: i.e. pivot table
 - Roll-up: $\downarrow$ granularity, higher lvl
 - Drill-down: $\uparrow$ granularity, lower lvl
 - Slice: equality condition on dimension
 - manage processed data
 - slice: fix single dim at 1 specific value
 - time-series analysis good
 - mental model of multidimensional tensor

- Star schema
 - 1 fact table = have FK to dim. table's PK w/ measurable, quantitative
 - many dim tables = has attr. of that dim's functions as explicit hierarchy

- Snowflake schema
 - dimension tables connected to other dims

- MOLAP (Multidim. OLAP) - static data
 - Data cube as multi-dim array - same queries
 - Optimized multi-dim array, not dense
 - Pre compute materialized slices, cross-tabs, diff cube granularities
 - fast response, expensive ETL time

- ROLAP (Relational OLAP) - large dim.
 - Data cube as star/snowflake, RDBMS
 - implemented in SQL (WHERE, GB, CUBE/ROLLUP)
 - no precomputation, SQL for materialize views
 - SQL tools dynamic data, ad-hoc queries

- HOLAP (hybrid OLAP) - high sparsity, flexible
 - some in relational, in specialized storage

- Spreadsheet
 - Workbook > sheets > cells > formula/value
 - rectangular ranges: B1:C3 $\leftarrow$ sheet #, row #, col #
 - entire column: F:F
 - reference stays anchored
 - A1 = moves: relative reference, follow along
 - \$A1 = lock col. A, row can change, drag right won't change A
 - A\$1 = lock row 1, col can change, drag down won't change 1
 - \$A\$1 = lock both = absolute referencing
 - so start of range fixed, while rest expands
 - constant lookup range to search same table
 - from this sheet
 - Sales! A:A = col. A on sales
 - XLOOKUP (search_key, lookup_range, result_range)
 - search_key = what you're looking for
 - lookup_range = where you're looking, usually col of diff table
 - result_range = where to get result, usually col of lookup-range table
 - XLOOKUP = where to put result
 - Direct manipulation: cont. representation, physical, visible
 - COUNTIF (range, condition)
 - (B2:B2) $\rightarrow$ range starts at second row
 - $\rightarrow$ B2 will change when you drag down
 - $\rightarrow$ B2 - how many times will we find B2 in range?

- Graph Databases + Knowledge Bases
 - Property Graphs
 - nodes: many entity types, variety of properties per node
 - edges: directed, variety of types of relationships + properties per edge, many relationship types
 - easy support of graph expansion/evolution/traversal/path queries, (flexible ordering) $\leftarrow$ JSON messy
 - declarative query language Cypher
 - Triple Stores
 - dfs for semantic queries
 - entities as triples (subject, predicate, object)
 - can represent property graphs
 - RDF = resource description format for triples
 - ontologies/knowledge bases
 - PE + TE good to encode knowledge
 - SPO & node-edge $\rightarrow$ node

- Create Graph
 - CREATE TABLE nodes
 - (node_id INTEGER PRIMARY KEY, type STRING, properties JSON)
 - CREATE TABLE edges
 - (edge_id INTEGER PK,
 - head_id INTEGER REFERENCES nodes (node_id),
 - tail_id "
 - type STRING, properties JSON)

- Recursive Traversal
 - WITH RECURSIVE in_usa (node_id) AS (
 - (SELECT node_id FROM nodes WHERE properties->>'name' = 'United States')
 - UNION (
 - SELECT tail_id FROM edges JOIN in_usa ON edges.head_id = in_usa.node_id WHERE edges.type = 'within'
 - base
 - recursive
 - SELECT * FROM in_usa
 - $\rightarrow$ get JSON value
 - $\rightarrow$ get it as text

⑬ Recursive CTE

WITH RECURSIVE t(n) AS (
VALUES (1) -- Base
UNION ALL
SELECT n+1 FROM t -- recursive case
WHERE n < 100 -- terminal cond.
)
SELECT SUM(n) FROM t
→ sum #s 1 to 100

⑭ MongoDB

- DB = DB
relation = collection
row = document; column = field
- Retrieval Queries (selection + projection)
 - db.collection.find(predicate, projection(optional))
 - exclude col w/ 0; include w/ 1
ex: _id: 0
 - default id specified
- Aggregation Queries (Pipeline)
 - dot notation to traverse trees
 - \$ special keyword (\$match)
 - bool or range match (\$gte)
 - accessing fields in agg pipelines \$fieldName
- syntax
 - db.table.aggregate([
 { \$group: { _id: "field to group", newField: { \$aggfunc: "\$field" } } }
 { \$match: { column: { \$gte: 150000 } } } ← having / where
 { \$sort: { field: -v1 } }
 { \$project: { field: 0 / 1 } }
 { \$limit: k }
])
 - \$all if all entries in array satisfy condition
 - \$in if a value present in array
 - \$exists if a field exists
 - \$unwind expand array → docs one per elem of array
 - \$lookup left outer equi join
 - \$insertMany & \$updateMany insert/update doc w/ \$set
- Examples
 - all paintings in db: db.paints.find({})
 - artists born later: db.artist.find({"bgdInfo.birthYear": {"\$gt": 1900}})
 - specific time period:


```
db.paints.find({ "$or": [ { "timeperiod": "Baroque" }, { "timeperiod": "Romantic" } ] })
```
 - db.paints.find({"mediums": "canvas"})
 - db.artist.find({"timeperiod": {"\$in": ["Baroque", "Romantic"]}})
 - db.paints.aggregate([
 { \$sort: { year: 1 } }
 { \$group: { _id: "\$artistId", firstPaintingName: { \$first: "\$paintingName" } } }
])
 ← 1st painting by yr per artist

• Lookup

↳ paintings by Netherland artists + project painting name + its country
 db.paintings.aggregate([
 { \$lookup: {
 from: "artists",
 localField: "artistId",
 foreignField: "artistId",
 as: "artistInfo"
 } },
 { \$unwind: "\$artistInfo" },
 { \$match: { "artistInfo.bgdInfo.country": "Netherlands" } },
 { \$project: { _id: 0, paintingName: 1, artistCountry: "\$artistInfo.bgdInfo.country" } }
])

⑮ Entity Resolution

- partition data into classes matching real world entities
- by reference sheet or heuristics (blocking + clustering/matching)
- Levenshtein Dist: count # of insert, delete, replacement btwn strings
- Blocking: group data into (overlapping) subsets, aka blocks
 - q-gram: block of substring of length q,
 - n-q+1 → w/o \$
 - small q → low precision (most entities in most blocks)
 - big q → low recall (entities not together as it should)
 - matching: w/in each block, match equiv. entities

```

Count # paintings per medium + avg purchase price
of paintings for each medium + oldest painting's
year per medium

db.paintings.aggregate([
    { $unwind: "$mediums" },
    { $group:
        { _id: "$mediums",
          count: { $sum: 1 },
          avgPurchasePrice: { $avg: "$purchasePrice" },
          oldestYear: { $min: "$year" } } },
    { $project:
        { _id: 0,
          medium: "$_id",
          count: 1,
          avgPurchasePrice: 1,
          oldestYear: 1 } }
])
    
```

• UPDATE

```

db.paintings.updateMany(
    { "purchasePrice": { $lt: 500000 } },
    { $inc: { "purchasePrice": 100000 },
      $set: { "status": "appreciated" } }
)
    
```

⑯ Window Functions

<window or agg-func> OVER (
[PARTITION BY <...>]
[ORDER BY <...>]
[RANGE BETWEEN <...> AND <...>]
)

- calculate running success rate

```

SELECT climberId, attempt_date,
    AVG(CASE WHEN is_successful = TRUE THEN 1 ELSE 0 END)
    OVER (PARTITION BY climberId ORDER BY attempt_date)
    AS running-success-rate
FROM attempts
ORDER BY climberId, attempt_date
    
```

- find all attempts where next attempt had

```

SELECT ...,
    LEAD(a.id, 1) OVER PARTITION BY a.climberId
    ORDER BY a.attempt_date AS next_attempt_id,
    LEAD(r.difficulty, 1) OVER (PARTITION BY a.climberId
    ORDER BY a.attempt_date) AS next_difficulty
FROM ...
    
```

- impute most recent successful climb

```

SELECT ...,
    CASE
        WHEN c.level IS NOT NULL THEN c.level
        ELSE (
            SELECT COALESCE_agg(r.difficulty ORDER BY a.attempt_date DESC)
            FROM attempts a
            JOIN routes r ON a.route_id = r.id
            WHERE a.climberId = c.id AND a.is_successful = TRUE
        )
    END AS imputed_level
    
```

⑰ Recursive

- all places reachable from A + total miles along each path

WITH RECURSIVE paths(dst, total_miles) AS (
 SELECT dst, miles
 FROM roads
 WHERE src = 'A' -- base: one-edge paths
 UNION ALL
 SELECT r.dst, p.total_miles + r.miles -- accumulator
 FROM paths p
 JOIN roads r ON r.src = p.dst -- step: extend path by 1 edge
)
SELECT * FROM paths

⑱ Map-Reduce (word count)

```

def map_fn(document: str) → dict[str, int]:
    word_count = defaultdict(int)
    for word in document.split():
        word_count[word] += 1
    return word_count

def reduce_fn(word: str, counts: list[int]) → dict[str, int]:
    return {word: sum(counts)}
    
```

- hello world
- hello world mapreduce
- mapreduce example
- hello hello

→ 4 mappers + outputs

- {hello: 1, world: 1}
- {hello: 1, world: 1, mapreduce: 1}
- {mapreduce: 1, example: 1}
- {hello: 3}

→ shuffle for reducer input

- hello: [1, 1, 0, 3]
- world: [1, 1, 0, 0]
- mapreduce: [0, 1, 1, 0]
- example: [0, 0, 1, 0]

→ reducer output

- {hello: 5}
- {world: 2}
- {mapreduce: 2} ← example: 1

To get output freq(w) = count(w)
 ① normal word count
 ② total word count
 • (word, count) → (TOTAL, count)
 • sum → (TOTAL, total-words)
 ③ compute rel. freq.
 • join counts
 (word, count/total-words)
 • collect output per word